System Architecture Document

1. System Overview

1.1 What Services Exist

Service	Technology	Role
Backend API / Application Server	Laravel 13.7 / PHP 8.3	Business logic, database access, authentication, authorization
Frontend SPA	React 19 + TypeScript + Vite 8	UI rendering, client-side routing, Inertia page hydration
Database	SQLite 3	Persistent storage for all application data
Queue / Job Processor	Laravel Queue (database driver, SQLite `jobs` table)	Async email delivery, background processing
File Storage	Local filesystem (`storage/app/public/`)	Document uploads (PDFs) and versioned snapshots
Scheduled Tasks	Laravel Scheduler (CRON)	Daily status updates and expiration reminders
Build / Bundler	Vite 8	Frontend asset bundling and HMR development

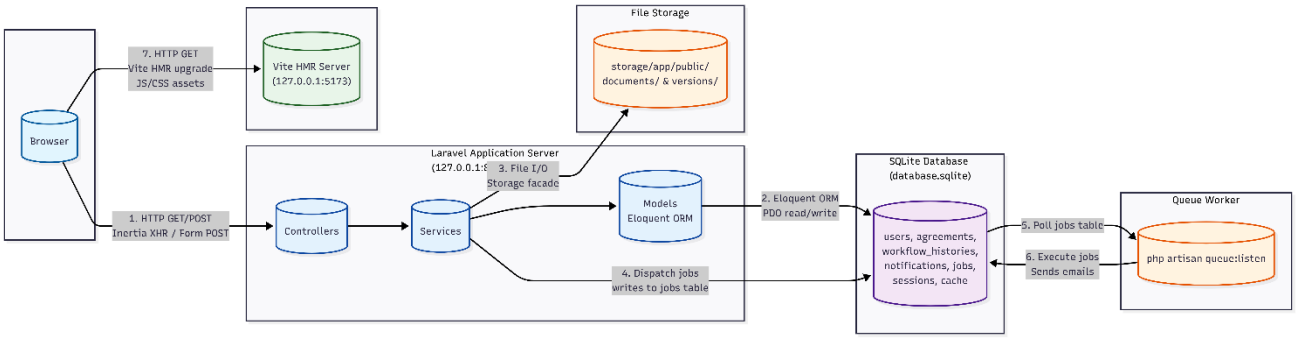
---

1.2 What Each Service Owns

Service	Data Owned	Business Logic	UI
Laravel Backend	All tables: `users`, `agreements`, `agreement_versions`, `workflow_histories`, `notifications`, `sessions`, `jobs`, `cache`, `passkeys`, etc.	Workflow state machine (`AgreementWorkflowService`), document storage, auth enforcement, scheduled commands, observer logic	None — server-renders Inertia page props only
React Frontend	Ephemeral client state (React hooks, Inertia page props)	Minimal — frontend workflow map (`lib/agreement.ts`) for UI helpers only	All React page components, layouts, shadcn/ui primitives
SQLite	All persistent application data, sessions, queues, cache	None	None
Vite	Build cache (`node_modules/.vite/`)	None	Bundled static assets served to browser

---

1.3 How They Communicate



Communication protocol summary:

- **Browser ↔ Laravel**: Standard HTTP (no WebSocket, no GraphQL, no REST API in the traditional sense)
- **Laravel ↔ SQLite**: PDO via Laravel's database abstraction (Eloquent ORM)
- **Laravel ↔ Filesystem**: PHP native file I/O via `Storage` facade
- **Laravel Queue Worker ↔ SQLite**: Polls `jobs` table for dispatchable jobs
- **No inter-service network calls** — all services run in a single process tree on one host

---

2. Integration Pattern

2.1 Primary Pattern: **Monolithic Application (Point-to-Point)**

This is a **monolithic** single-deployable application. All business logic lives in one Laravel application process. The "frontend" is not a separate service — it is embedded inside Laravel via Inertia.js and runs as a React SPA in the browser.

Point-to-point calls dominate internally:

Controller → AgreementWorkflowService::forward()

→ AgreementDocumentStorageService::store()

→ Model (Eloquent)

→ SQLite

There is no message broker, no event bus, no service mesh.

2.2 Secondary Pattern: **Event-Driven (Eloquent Observers)**

The AgreementObserver reacts to Eloquent lifecycle events:

Agreement::saving() → Agreement::syncStatus() // Auto-recomputes status on every save

This is an event-driven element within an otherwise synchronous monolith.

2.3 Pattern: **Synchronous Request-Response (Inertia)**

Every user action (click, form submit) results in a full page navigation or an Inertia XHR request. The server synchronously renders the next page state and returns JSON props. The browser hydrates the React component. There is **no async frontend rendering**.

2.4 Justification

- **Monolithic simplicity**: The application scope is a single-tenant internal tool. Splitting into microservices would add operational complexity without benefit.
- **Inertia over REST**: The app has no mobile clients and no third-party consumers. Inertia's server-rendered SPA model eliminates the need for a separate API layer, reducing serialization overhead and making server-side rendering trivially easy.
- **Database queue over Redis**: The queue uses the `database` driver backed by SQLite's `jobs` table. This avoids adding a Redis dependency. Throughput is low (internal tool, few concurrent users), so Redis would be premature optimization.
- **No WebSocket**: Notifications are pulled on page load (shared via `HandleInertiaRequests` middleware). Real-time push is unnecessary for the workflow cadence.

---

3. Data Contracts

Contract 1: Create Agreement

Field	Value
Method + Path	`POST /agreements`
Content-Type	`multipart/form-data`
Auth	`auth` + `role:coordinator,admin`

Request Body:

Field	Type	Required	Validation
`title`	string	Yes	max:255
`type`	enum	Yes	one of: `MOA`, `MOU`
`partner_organization`	string	Yes	max:255
`partner_organization_id`	integer	No	exists:partner_organizations,id
`description`	string	No	max:5000
`signed_at`	date	No	date, before or equal:expires_at
`expires_at`	date	No	date, after or equal:signed_at
`document`	file	No	mimes:pdf, max:10240
`status`	enum	No	one of: `draft`, `for_review` (default: `for_review`)

Success Response: `302 Redirect` to `GET /agreements/{id}`

Error Responses:

- `403 Forbidden` — User lacks `coordinator` or `admin` role
- `422 Unprocessable Entity` — Validation failure

Contract 2: Forward Agreement Workflow

Field	Value
Method + Path	`POST /agreements/{id}/forward`
Content-Type	`application/x-www-form-urlencoded` or `multipart/form-data`
Auth	`auth` + `role:coordinator,admin`

Request Body:

Field	Type	Required	Validation
`next_status`	string	Yes	Must equal `AgreementWorkflowMap::nextStatus(current)`
`remarks`	string	No	max:1000
`version_id`	string	No	exists:agreement_versions,id
`target_user_id`	integer	No	exists:users,id

> *The `next_status` is validation-enforced server-side — users cannot choose an arbitrary stage. The forward action is deterministic based on the current workflow state.*

Success Response: `302 Redirect` back to referer

Error Responses:

- `403 Forbidden` — User role not authorized OR coordinator not at correct `coordinator_stage`
- `422 Unprocessable Entity` — Invalid `next_status` for current stage
- `404 Not Found` — Agreement does not exist

Server-side side effects:

- `workflow_status` updated to `next_status`
- `WorkflowHistory` record created
- `Notification` records created for target role users at next stage
- If `next_status === 'active_agreement'`: `signed_at` set to now() if null, `status` set to `active`

Contract 3: Return Agreement

Field	Value
Method + Path	`POST /agreements/{id}/return`
Content-Type	`application/x-www-form-urlencoded`
Auth	`auth` + `role:coordinator,admin`

Request Body:

Field	Type	Required	Validation
`remarks`	string	Yes	max:1000
`return_to`	string	No	Must be a valid workflow stage string

Success Response: `302 Redirect` back to referer

Error Responses:

- `403 Forbidden` — User cannot return from current stage
- `404 Not Found` — Agreement does not exist

Server-side side effects:

- If `from === 'legal_assistant_ii'`: agreement returns to `draft`, notification to original `submitted_by`
- Otherwise: agreement returns to `previousStatusForReturn(from)`, notifications to target stage coordinators

---

Contract 4: Subscribe to Expiration Notifications

Field	Value
Method + Path	`POST /agreements/{id}/subscribe`
Content-Type	`application/x-www-form-urlencoded`
Auth	`auth`

Request Body: Empty (agreement ID is path parameter)

Success Response:

```
{   "ok": true,   "subscribed": true }
```

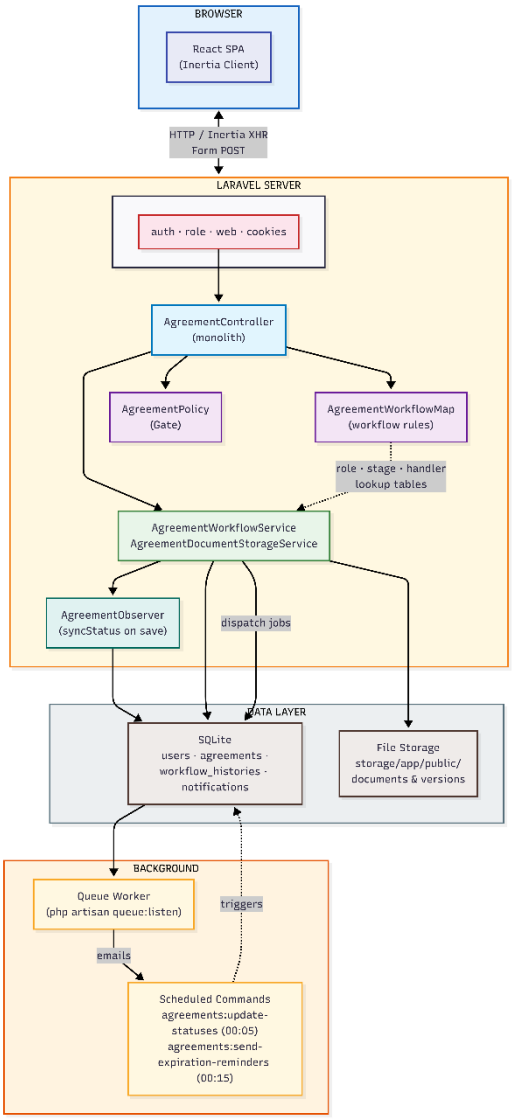
Error Responses:

- `403 Forbidden` — User cannot view this agreement (`AgreementPolicy::view()` fails)
- `404 Not Found` — Agreement does not exist

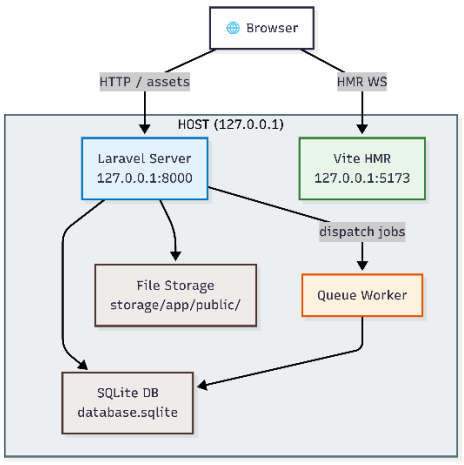
---

4. Architectural Viewpoints

4.1 Logical View (Component Diagram)



4.2 Deployment View (Development)



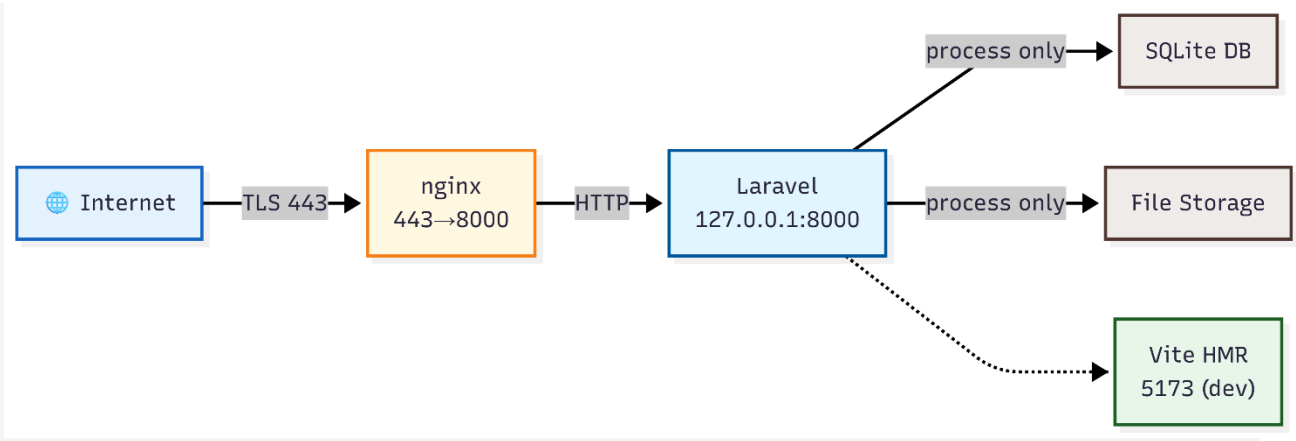
No Docker. Services are OS processes on a single host:

Port	Service	Publicly accessible?
`8000`	Laravel dev server	No (localhost only)
`5173`	Vite HMR	No (localhost only)

---

5. Security Considerations

5.1 Trust Boundaries



Zone	Contents	Accessible from
Internet	External browsers / clients	Public internet
Reverse Proxy	nginx / Apache	Port 443/80 from internet
Public (localhost)	Laravel server, Vite (dev)	Only from inside host via proxy
Internal Only	SQLite, file storage, queue worker	**No network access** — PHP process only

5.2 Authentication End-to-End

```
1. User → GET /login  (Fortify Inertia page) 2. User → POST /login (credentials: email + password) 3. Fortify validates against users table (bcrypt hash) 4. Session created: encrypted cookie + sessions table (SQLite) 5. All subsequent requests: Cookie sent → auth middleware validates session 6. Optional 2FA: POST /two-factor-challenge (TOTP via Google Authenticator / passkey via WebAuthn) 7. Protected resource accessed
```

Auth mechanisms:

Mechanism	Implementation
Session auth	Laravel default (`config/session.php`, database driver)
Password hashing	`BCRYPT_ROUNDS=12` via Fortify
2FA	TOTP via `laravel/fortify` (`two_factor_secret`, `two_factor_confirmed_at` columns)
Passkeys	WebAuthn/FIDO2 via `@laravel/passkeys`, stored in `passkeys` table

Mechanism	Implementation
CSRF	Laravel token on all non-GET routes (Inertia sends `X-CSRF-TOKEN` header)
Rate limiting	Login: 5/min per IP+email; 2FA: 5/min per session; Passkey: 10/min

5.3 Sensitive Data Protection

Data	Storage	Protection
User passwords	`users.password` (bcrypt hash)	Never stored in plaintext
Session token	`sessions.cookie` (cookie) + `sessions.table` (encrypted in DB)	`SESSION_ENCRYPT=true`; cookie is signed
2FA secret	`users.two_factor_secret` (encrypted)	Laravel Fortify encryption
Passkey credentials	`passkeys.credential` (encrypted)	Fortify WebAuthn encryption
Uploaded PDFs	`storage/app/public/documents/` (filesystem)	Not web-accessible directly; served via `AgreementController::download()` with auth check
Notification content	`notifications.message`	Only visible to recipient (`user_id` filter)

5.4 Known Risks and Limitations

Risk	Severity	Mitigation (Future)
No HTTPS enforcement in dev	Medium	Production reverse proxy must enforce TLS. No `TrustProxies` or `forceScheme` middleware in code.
SQLite as queue backend	Low	Single-server deployment; database queue driver is acceptable for low throughput. Redis would be needed only at scale.
No row-level encryption	Low	Sensitive PDF content stored unencrypted on filesystem. Add LUKS or application-layer encryption for GDPR compliance.
No audit log immutability	Low	`workflow_histories` and `activity_logs` are regular Eloquent models — an admin could delete records. Add database triggers or write-to-append log.
CSRF meta tag exposed to XSS	Medium	The `csrf-token` meta tag could be read by XSS. CSP headers should be added to prevent inline scripts.
No API authentication layer	Low	Not applicable — the app has no external API consumers. All access is through the Inertia SPA.

Risk	Severity	Mitigation (Future)
`system_admin` role fallback	Medium	`RoleMiddleware` has a fallback granting access if user's name/role contains "system" when route allows `system_admin`. While intentional for seed data, it weakens role enforcement for any user with "system" in their name.

---

6. Architectural Tradeoffs

Tradeoff 1: SQLite over PostgreSQL/MySQL

Decision: Used SQLite (file-based) instead of a server-based RDBMS.

What was given up:

- **No concurrent writers**: SQLite uses file-level locking. Only one write transaction can execute at a time. High concurrent write throughput will cause `SQLITE_BUSY` errors.
- **No network access to DB**: The database is a file on the local filesystem. No remote connections, no connection pooling.
- **Limited to single-server deployment**: Cannot have a separate database host.

What was gained:

- **Zero configuration**: No DB server to install, start, or manage. `cp .env.example .env && php artisan migrate` just works.
- **Portable**: The entire application is a directory — `git clone` and run.
- **Simplified CI/CD**: No database service container to configure in test pipelines. PHPUnit uses `:memory:` SQLite.
- **Acceptable for scope**: This is an internal single-tenant application with a small user base. Concurrency is low; file-level locking is never contended.

Verdict: For a demo/single-developer internal tool, SQLite's limitations are never encountered. Scaling to a multi-user production deployment would require migration to PostgreSQL.

Tradeoff 2: Synchronous Inertia (Server-Rendered SPA) over REST + Client-Side Fetching

Decision: All data fetching uses Inertia's server-side rendering model instead of a REST API consumed by the frontend via `fetch` or React Query.

What was given up:

- **No API for mobile apps or third-party consumers**: The frontend and backend are tightly coupled. A mobile app or external integration would require building a separate API layer.
- **Full page states returned on every navigation**: Inertia sends the full page component tree and props on each navigation, even for small data fetches. This is larger payload than a targeted REST endpoint.

- **No granular caching strategies**: Cannot cache individual API responses; the entire page state is rendered server-side.

What was gained:

- **Eliminated an entire layer**: No API routes, no API controllers, no request/response DTOs, no API documentation, no Postman collections.
- **Type-safe routing**: Wayfinder plugin auto-generates typed route helpers from Laravel routes (`import { show } from '@/actions/AgreementController'`). No manual `route()` string calls.
- **Server-side rendering for free**: Every page is rendered on the server. No loading flicker, no SEO concerns (though irrelevant for an internal app), no client-side data fetching to manage.
- **Shared validation**: Form request classes validate server-side; the same validation cannot be accidentally skipped on the client.

Verdict: This is the correct tradeoff for a single-tenant internal tool with no API consumers. The "API" is the Laravel controller methods themselves, invoked directly by Inertia. Rebuilding this as a REST API + React app would double the codebase with no benefit.

Tradeoff 3: Database Queue Driver over Redis/Memory

Decision: Used Laravel's `database` queue driver backed by SQLite's `jobs` table instead of Redis or a memory-based driver.

What was given up:

- **Persistence over speed**: Jobs are serialized to SQLite and polled. There is latency between dispatch and execution (poll interval). Redis would be near-instant.
- **No distributed worker support**: Redis supports multiple workers across machines. SQLite `jobs` table with file locking cannot safely support multiple queue workers on different hosts.

What was gained:

- **No Redis dependency**: One fewer service to run, configure, and monitor.
- **Jobs survive crashes**: Unlike an in-memory queue, jobs persist across worker restarts.
- **Acceptable latency**: For email reminders (sent every 15 minutes by cron), sub-second queue latency is irrelevant.

Verdict: The `database` driver is the pragmatic choice for a single-server internal app. Redis would be added only when queue throughput demands it.